TECHNICAL DOCUMENTATION

RAG-Based Customer Support Assistant

1. Introduction

1.1 What is Retrieval-Augmented Generation (RAG)?

Retrieval-Augmented Generation (RAG) is a system design pattern that combines two distinct AI capabilities:

1. **Information Retrieval Systems**
    - Responsible for locating relevant pieces of information from a structured knowledge base
    - Operate using vector similarity (semantic search)
2. **Large Language Models (LLMs)**
    - Responsible for generating human-like responses
    - Capable of summarization, reasoning, and contextual understanding

A standalone LLM generates answers purely from learned patterns, which introduces two limitations:

- Knowledge is static (frozen at training time)
- It may hallucinate (generate incorrect but confident responses)

A standalone retrieval system, on the other hand:

- Can fetch relevant text
- But cannot synthesize or explain information

RAG combines both approaches:

- First retrieves relevant chunks from a document store
- Then injects them into the LLM prompt
- The LLM generates a response grounded strictly in retrieved content

This ensures:

- Higher accuracy
- Reduced hallucination
- Domain-specific reliability

1.2 Why RAG is Needed for Customer Support

Customer support systems require:

- High accuracy
- Context awareness
- Up-to-date information

Traditional approaches fail due to:

1. FAQ Systems

- Keyword-based matching
- Fail on paraphrased queries

Example:

- "Can I return my product?"
- "What is the refund policy?"
  → Same intent, different wording

2. Standalone LLM Systems

- May generate incorrect policy information
- Cannot guarantee factual accuracy
- No grounding in company documents

3. RAG Advantages

RAG solves these problems by:

- Mapping semantic queries to relevant knowledge
- Ensuring answers are based on actual documents
- Allowing easy updates (just change PDF)
- Improving trust through grounded responses

1.3 Use Case Overview

The system is designed for a customer support scenario where:

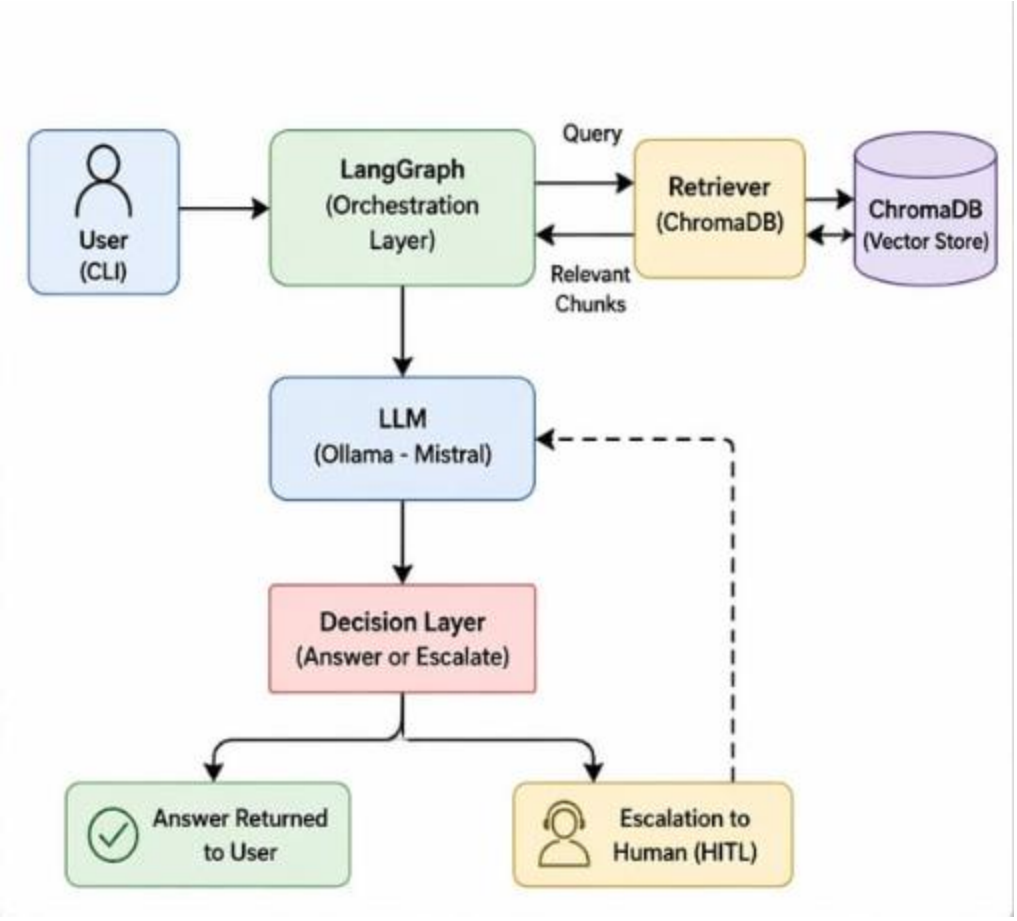
- A company maintains its policies in PDF format
- Users ask questions in natural language
- The system retrieves relevant content
- Generates answers using that content
- Escalates when required

Example:

Query	Behavior
“What is refund policy?”	Answer from document
“My order is delayed”	Escalate to human

---

3. System Architecture Explanation



The system operates in two major phases:

2.1 Ingestion Phase (Offline Processing)

This phase prepares the knowledge base for efficient retrieval.

Step 1: Document Loading

- PDF is loaded using PyPDFLoader
- Text is extracted page-wise
- Metadata (page number) preserved

Step 2: Text Chunking

The extracted text is split into smaller segments.

Configuration:

- Chunk size: 300 characters
- Overlap: 30 characters

Why Chunking is Required

- LLMs cannot process very large text directly
- Embeddings work better on smaller semantic units
- Improves retrieval precision

Step 3: Embedding Generation

Each chunk is converted into a vector representation using:

- Sentence Transformers (all-MiniLM-L6-v2)

Properties of Embeddings

- Numerical representation of meaning
- Similar meaning → similar vectors
- Enables semantic search

Step 4: Vector Storage (ChromaDB)

- Embeddings stored in ChromaDB
- Each vector linked with original text
- Enables fast similarity search

2.2 Query Phase (Online Processing)

This phase handles real-time user queries.

Step 1: Query Input

- User inputs query via CLI
- Query treated as natural language

Step 2: Query Embedding

- Query converted into vector
- Same model as document embeddings

Step 3: Retrieval

- Cosine similarity used
- Top-k = 2 relevant chunks retrieved

Why Top-k = 2?

- Reduces noise
- Keeps context concise
- Works well for small datasets

Step 4: Context Construction

- Retrieved chunks combined
- Forms input context for LLM

Step 5: LLM Processing

Context + Query → passed to
→ Ollama (Mistral)

LLM Role

- Understand context
- Generate structured response
- Follow prompt constraints

Step 6: Decision Logic

System evaluates response quality.

Step 7: Output

- If valid → return answer
- If invalid → escalate

2.3 Component Interaction Model

The system uses a **state-driven workflow** managed by
→ LangGraph

State Object

```
{
  "query": str,
  "answer": str,
  "decision": str
}
```

Flow

Each node:

- Reads state
- Processes data
- Updates state

---

3. Design Decisions

3.1 Chunk Size Selection

Options Considered:

Size	Pros	Cons
128	Precise	Loses context
300	Balanced	Slight overlap needed
1000+	Rich context	Poor embeddings

Final Choice:

- 300 size
- 30 overlap

Reason:

- Maintains semantic meaning
- Improves retrieval
- Efficient for LLM

3.2 Embedding Strategy

- Same model for:

- Query
- Documents

Reason:

- Consistent vector space
- Accurate similarity

3.3 Retrieval Approach

- Cosine similarity
- Top-k retrieval

Trade-off:

Factor	Decision
Speed	High
Accuracy	Moderate
Complexity	Low

3.4 Prompt Design

Key Constraints:

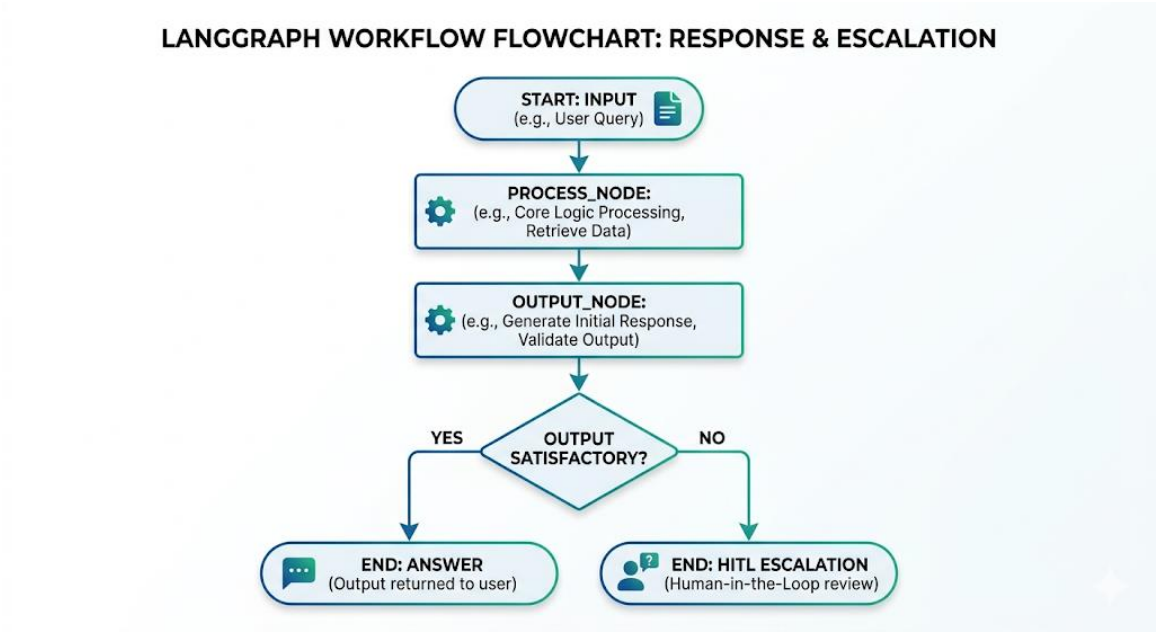
- "Answer ONLY using context"
- "If not found → NOT_FOUND"

Benefit:

- Prevents hallucination
- Ensures grounded output

---

4. Workflow Explanation — LangGraph



4.1 Why Graph-Based Workflow

Traditional pipeline:

input → retrieve → generate → output

Limitation:

- No branching
- No decision logic

4.2 LangGraph Solution

Graph-based execution allows:

- Conditional routing
- State tracking
- Modular nodes

4.3 Nodes

process_node

- Retrieval
- LLM call

output_node

- Decision
- Output

4.4 Workflow

START → process_node → output_node → END

4.5 State Transitions

Stage	State Update
Input	query
Process	answer
Output	decision

---

5. Conditional Logic — Routing

5.1 Decision Criteria

The system checks:

- Missing context
- Weak response
- Invalid answer

5.2 Escalation Conditions

- “NOT_FOUND” present
- Response too short
- No relevant chunks

5.3 Logic Example

```
if "not_found" in response or len(response) < 40:
    decision = "ESCALATE"
```

---

6. HITL Implementation

6.1 Role

Handles:

- Unknown queries
- Low confidence answers

6.2 Flow

1. Detection
2. Escalation
3. Human input
4. Final response

6.3 Implementation

```
human = input("Human Agent: ")
```

6.4 Benefits

- Improves reliability
- Prevents incorrect answers

6.5 Limitations

- Slower
- Depends on human

---

7. Challenges & Trade-offs

Challenge	Trade-off
Retrieval	Speed vs accuracy
Chunking	Context vs precision
LLM	Cost vs performance
HITL	Accuracy vs latency

---

8. Testing Strategy

8.1 Testing Levels

- Unit testing
- Integration testing
- End-to-end testing

8.2 Test Cases

Type	Query	Expected
Valid	Refund policy	Answer
Invalid	Rockets	Escalate

Edge	Empty query	Handle safely

8.3 Evaluation

- Accuracy of responses
- Correct escalation
- Stability

---

9. Future Enhancements

9.1 Multi-Document Support

- Handle multiple PDFs

9.2 UI Integration

- Web-based interface

9.3 Memory

- Context-aware conversations

9.4 Advanced Retrieval

- Hybrid search
- Re-ranking

9.5 Deployment

- Cloud hosting
- API integration